ActPlane: OS-Enforced Agent Harnesses

Paper #XXX
Anonymous Submission

Abstract

AI coding agents require harnesses to enforce behavioral contracts for effective and safe operation: do not leak secrets, run tests before committing, only write to tmp dir. Agents operate at three levels: *intent* (semantic goals and reasoning), *action* (tool calls), and *behavior* (OS operations: file access, process execution). Our empirical study of 64 projects shows that while project harness instructions are declared in natural language, 63% consist of behavioral contracts, 80% of which involve system-level behavior, and 78% of projects require cross-event state tracking.

Yet existing agent harness enforcement leaves an *intent–behavior gap*: intent-level prompts are forgettable and probabilistic; action-level guards check tool calls but miss effects that reach the OS through subprocesses or information flow; behavior-level sandboxing is unbypassable but disconnected from intent, making it unsuitable for harness enforcement.

ActPlane is a programmable OS-level control plane for agent harnesses that bridges the intent–behavior gap: agents declare behavioral contracts in a DSL; ActPlane persists them as in-kernel labeled information-flow rules, observing and enforcing agent behavior across process, file, and network boundaries, and providing semantic feedback. We implement ActPlane with eBPF and evaluate on the contracts from the empirical study, showing that ActPlane can enforce complex cross-event contracts with low overhead and provides actionable feedback to agents.

CCS Concepts: • **Computer systems organization** → *Embedded and cyber-physical systems*; • **Security and privacy** → *Software and application security*; • **Software and its engineering** → *Software safety*.

Keywords: AI agents, eBPF, BPF-LSM, labeled information-flow control, information-flow control, provenance, semantic feedback

ACM Reference Format:
Paper #XXX. 2026. ActPlane: OS-Enforced Agent Harnesses. In *Proceedings of 2026 ACM SIGOPS Annual Technical Conference (ATC '26).* ACM, New York, NY, USA, 7 pages.

1 Introduction

AI coding agents have become widely deployed tools for software development [12]. They operate as long-running processes with access to shells, source trees, package managers, and external APIs, autonomously writing code, running tests, and managing infrastructure across extended sessions.

As agents gain autonomy, they require harnesses to enforce behavioral contracts. Projects encode such contracts in instruction files such as `CLAUDE.md` and `AGENTS.md` [5, 15]. Because LLM compliance is inherently probabilistic [14], enforcement cannot reside inside the model [3, 25].

Existing agent harness enforcement operates at a single level, leaving an *intent–behavior gap*. An agent's execution spans three abstraction levels: *intent* (what the agent declares it will and will not do), *action* (the tool calls it issues through its runtime), and *behavior* (the actual OS operations, including `open`, `execve`, and `connect`, that result) [12]. The mapping from action to behavior is many-to-many and opaque: a single `run_command("make")` can trigger hundreds of system calls, and the same `git push` can be reached through a direct tool call, a `bash -c` string, a Python subprocess, or a compiled helper. Intent-level enforcement (prompt instructions) is probabilistic. Action-level enforcement (tool-call guards [22, 26]) is deterministic but bypassable: it checks tool calls, not the OS behavior they produce. Behavior-level enforcement (kernel-level enforcers such as CamQuery and Tetragon [6, 18]) is unbypassable but returns failures disconnected from intent. No single level bridges the gap.

Real behavioral contracts frequently require cross-event information-flow tracking that no single enforcement level provides. A contract like "never expose secrets to the network" is not a predicate on one system call; it requires knowing that the current process previously read a secret file and now attempts a network connection. A contract like "run tests before committing" requires knowing that a test process executed before a commit process in the same session. A corpus study of instruction files from 50 popular projects confirms that such cross-event, cross-temporal contracts are pervasive, yet today's agent harnesses [3, 25] keep their enforcement and feedback components at the application layer, where they cannot connect behavior to intent.

ActPlane is a programmable OS-level control plane for agent harnesses that bridges intent and behavior. Agents and developers declare behavioral contracts in a compact DSL, a form of voluntary confinement analogous to `pledge()` in OpenBSD [17]. ActPlane compiles these declarations into labeled information-flow rules and loads them into an eBPF/BPF-LSM backend. Named labels propagate across process, file, and network objects at kernel hooks, encoding execution history as checkable state: when a process reads a file labeled `SECRET`, the process acquires that label; when it forks, the child inherits it. When a labeled subject violates a rule, ActPlane returns semantic feedback to the agent, specifying which contract was violated, why, and what alternative path satisfies it. ActPlane serves as the control plane for the harness: it compiles intent-level contracts into behavior-level

rules (the policy path) and translates behavior-level violations back into intent-level feedback (the feedback path); the eBPF kernel hooks form the data plane. Each rule independently selects audit (observe and notify), block (pre-operation denial), or kill (post-operation termination), so teams can deploy contracts incrementally—auditing first, then enforcing once the rules are validated. Unlike prior work that enforces at a single layer (§7), ActPlane unifies cross-event information-flow observation and enforcement with semantic feedback at the OS level, where contracts survive context loss and cannot be bypassed below the tool API.

This paper makes three contributions.

1. It defines the *intent–behavior gap* for AI agents and grounds it with a corpus study of instruction files from 50 popular projects, showing that cross-event information-flow contracts are pervasive in real agent instruction files.
2. It presents ActPlane, a programmable OS-level control plane for agent harnesses that compiles agent-declared behavioral contracts into eBPF/BPF-LSM labeled information-flow rules, observing and enforcing agent behavior across process, file, and network boundaries and providing semantic feedback that reconnects behavior-level violations to intent-level remediation.
3. It evaluates ActPlane on bypass coverage across five tool paths, agent recovery rate under four feedback conditions, false positives under benign workloads, and per-event overhead, comparing against action-level and behavior-level baselines.

2 Background and Motivation

2.1 Agent Harnesses and Behavioral Contracts

An agent harness is the non-model infrastructure that mediates between an LLM agent and its execution environment [3, 25]. Harness components include tool dispatch, memory management, orchestration, enforcement, and feedback. This paper focuses on the enforcement and feedback components: the mechanisms that ensure an agent's behavioral contracts hold at runtime.

Behavioral contracts are constraints that govern an agent's observable effects rather than its internal reasoning. Projects encode them in instruction files such as `CLAUDE.md` and `AGENTS.md` [5, 15]: do not push directly to `main`; run tests before committing; never expose secrets to the network. These contracts differ from coding-style directives (e.g., "prefer `const` over `let`") in that they produce observable effects at the system-call boundary.

2.2 The Intent–Behavior Gap

An agent's execution spans three abstraction levels.

Intent is the agent's own goals and self-constraints, i.e., what it declares it will and will not do. **Action** is the tool calls the agent issues through its runtime (`read_file`, `run_command`, `edit_file`), mediated by the agent framework.

Behavior is the actual OS-level operations that result: `open`, `execve`, `connect`, and `fork`.

The mapping from action to behavior is many-to-many and opaque. A single `run_command("make")` triggers hundreds of system calls. The same `git push` can be reached through a direct tool call, a `bash -c` string, a Python `subprocess`, or a compiled helper binary. This opacity creates an *intent–behavior gap*: contracts declared at the intent level cannot be reliably enforced at a single intermediate level.

Enforcement placed at the intent level (prompt instructions) is probabilistic: LLM compliance degrades with context length and task complexity [14]. Enforcement placed at the action level (tool-call guards [22, 26]) is deterministic but bypassable: it intercepts tool calls, not the OS behavior they produce. Enforcement placed at the behavior level (syscall filters, sandboxes [6, 18]) is complete but returns opaque `-EPERM` failures with no connection to the agent's declared contracts. Each level addresses a different part of the stack; none spans the full gap from intent to behavior.

A cooperative agent does not intentionally bypass tool-layer guards, but the opacity of the action-to-behavior mapping causes unintentional bypass in routine operation. Three representative scenarios illustrate this. (1) An agent calls `run_command("npm test")`; the test harness internally invokes `git push` to a staging server. The tool-call guard sees `npm test` (a permitted action), while the kernel observes the forbidden `git push` behavior. (2) An agent calls `read_file(".env")` to inspect a configuration value, then later calls `run_command("curl api.example.com")`. Each tool call is individually permitted, but the combination constitutes secret exfiltration, a cross-event violation visible only to an enforcer that tracks information flow from file read to network connect. (3) An agent writes a Python script containing `subprocess.run(["git", "push", "-force"])` and executes it. The tool-call guard sees `run_command("python script.py")`; the kernel observes the nested `execve("git", ["push", "-force"])`.

2.3 Threat Model

ActPlane targets a *cooperative but probabilistic* agent. The agent is not adversarial: it does not attempt to defeat the enforcement mechanism through obfuscation or exploitation. However, because LLM inference is inherently stochastic, the agent may violate any given contract with non-zero probability at each step, regardless of context length or prompt design. Over a long session with many steps, the cumulative probability of at least one violation approaches certainty.

The trust boundary is the kernel. The eBPF programs and BPF maps that propagate labels and check rules run in kernel space and cannot be modified by the agent. The policy DSL and feedback formatting run in a trusted userspace process (`actplane run`) that is not part of the agent's process tree. The monitored domain is the agent's entire process tree,

including all descendants spawned through tools, shells, or subprocesses.

ActPlane does not defend against an agent that deliberately exploits kernel vulnerabilities or manipulates the eBPF loader. Such attacks require `CAP_BPF` or root privileges that the agent does not hold. Side-channel inference (deducing policy rules from `-EPERM` patterns), deliberate label pollution (triggering over-tainting to cause denial of service), and feedback-oracle attacks (using remediation strings to restructure an approach toward a forbidden goal through compliant steps) all require adversarial intent and are outside this threat model.

3 Corpus Study

This section characterizes behavioral contracts in real agent instruction files and identifies the subset that requires cross-event information-flow enforcement.

3.1 Methodology

We collected `CLAUDE.md` and `AGENTS.md` files from 50 popular open-source projects, selected by GitHub star count and filtered to exclude documentation-only repositories. From these files we extracted 866 candidate imperative statements using keyword matching ("never," "must," "do not," "before," "only") and manually coded each statement along seven dimensions: (D1) coding style vs. behavioral contract, (D2) contract category, (D3) observable at the syscall boundary, (D4) expressible in the ActPlane DSL, (D5) enforceable without false positives under normal use, (D6) robust to naming and path variation, and (D7) temptable (likely to be violated by an agent in a realistic task). Two annotators independently coded the full set; we report Cohen's κ for inter-rater reliability.

3.2 Per-Action vs. Cross-Event Contracts

Behavioral contracts fall into two categories that differ in enforcement requirements.

Per-action contracts constrain a single operation independently of execution history. Examples include "do not execute `rm -rf`" and "do not run `git branch`." These contracts can be enforced by matching a single system call or tool call against a static rule.

Cross-event contracts constrain an operation based on prior interactions with other objects. Examples include "a process that has read `.env` must not connect to an external endpoint" (data-flow contract) and "commit only after tests have passed" (temporal contract). These contracts require tracking information flow or execution ordering across process, file, and network boundaries.

3.3 Implications for Enforcement

Per-action contracts can be enforced by per-event matching at any level: tool-call guards, seccomp filters, or eBPF kprobes. Cross-event contracts cannot, because they depend on state accumulated across multiple operations and objects. Enforcing them requires a mechanism that (1) tracks provenance across process, file, and network boundaries, (2) encodes that provenance as checkable state at each enforcement point, and (3) connects violations back to the declared contract for semantic feedback.

Labeled information-flow control satisfies all three requirements. Labels encode cross-event provenance as per-node bitmasks, propagation rules maintain the invariant across fork, exec, read, write, and connect operations, and source-to-sink rules check the accumulated state at each enforcement point. The ActPlane DSL compiles cross-event contracts directly into this model.

```
actplane.yaml
  policy: |
        |
        v
Rust compiler ------> struct taint_config
        |                    |
        v                    v
 actplane run -----> eBPF loader + BPF maps
        |                    |
        v                    v
  target command <--- kernel hooks
        |
        v
.actplane/last-violation.txt
```

Figure 1. ActPlane pipeline. The Rust compiler lowers the policy DSL into a fixed-size binary configuration. The eBPF loader writes it into read-only data, attaches to kernel hooks, and reports violations as NDJSON. The runner maps rule identifiers back to human-readable feedback.

4 Design

ActPlane is a programmable OS-level control plane for agent harnesses. It compiles behavioral contracts into labeled information-flow rules at OS kernel hooks, observing and enforcing agent behavior and returning semantic feedback when a rule is violated. Figure 1 summarizes the pipeline.

4.1 Policy Model

ActPlane policies are object-level information-flow rules over three node types: processes (identified by pid and command name), files (keyed by path hash), and endpoints (keyed by IPv4 address and mask). The model operates at object granularity rather than byte granularity [13], trading precision for low-overhead whole-system coverage.

4.1.1 Labels and Propagation. Each node carries a 64-bit label bitmask. A *source* declaration introduces labels: an exec source labels processes that execute a matching binary, a file

```
label AGENT
source SECRET = file "**/.env"

rule no-secret-egress:
  deny connect endpoint "*" if SECRET
  effect block
  reason "secret-derived data must stay local"
  remediation "run the redactor first"

rule no-agent-git:
  deny exec "**/git" if AGENT
  effect kill
  reason "agent must not invoke git directly"
```

Figure 2. Two ActPlane contract rules. The first uses BPF-LSM pre-operation denial; the second uses harness-level termination.

source labels files matching a path pattern, and an endpoint source labels network endpoints matching an address range. Labels propagate through fixed transfer functions:

Let $L(x)$ denote the label bitmask of node x. Labels propagate through fixed transfer functions:

- **fork**$(p \to c)$: $L(c) := L(c) \cup L(p)$.
- **exec**(p, f): $L(p) := L(p) \cup L(f) \cup S(f)$, where $S(f)$ is the set of source labels matching f.
- **read**(p, f): $L(p) := L(p) \cup L(f)$.
- **write**(p, f): $L(f) := L(f) \cup L(p)$.
- **connect**(p, e): $L(e) := L(e) \cup L(p)$.

All transfer functions are monotonic: labels accumulate and are never removed by propagation. The only mechanism that removes a label is an explicit *declassify* directive in the DSL, which clears specified labels when the agent executes a designated gate binary (e.g., a redaction tool). This monotonicity bounds over-tainting: a node's label set grows at most to the union of all source labels reachable through observed flow edges.

These transfer functions maintain a provenance invariant: if a process p carries label ℓ, then information from some source tagged with ℓ has reached p through observed OS-level flow edges.

4.1.2 Rules and Effects.

A rule contains one or more *deny* clauses. Each clause specifies an operation (`exec`, `read`, `write`, `unlink`, `connect`, or `open`), a target pattern, a Boolean expression over required and forbidden labels, and an optional condition (target allow-list, lineage gate, or temporal after-gate).

Each rule declares an explicit effect that determines how the kernel responds to a match.

Audit: the kernel emits a violation event and allows the operation to proceed.

Block: a BPF-LSM hook returns `-EPERM` before the operation commits. Block is available only when the `bpf` LSM is active in the running kernel; it is not silently downgraded.

Kill: the kernel sends `SIGKILL` to the violating task. From a BPF-LSM hook, kill also returns `-EPERM`; from a tracepoint, kill is post-operation termination.

When multiple rules match the same event, the strongest effect applies in the order kill > block > audit.

4.1.3 Contract Language Scope.

The DSL is not a general-purpose mandatory access control policy. Its primitives map to recurring agent workflow patterns: "after tests," "through a review tool," "data derived from untrusted input," and "read-only sub-agent." The grammar is small so that contract authors can translate natural-language instructions into source/sink/gate shapes without security-policy expertise.

4.2 System Architecture

4.2.1 Compiler and Kernel ABI.

The Rust compiler parses the DSL and lowers it into a C-compatible `struct taint_config`. Lowering allocates label bits, expands Boolean expressions into required and forbidden bitmasks, lowers glob patterns into exact, prefix, suffix, or wildcard match kinds, and compiles IPv4 patterns into network/mask pairs. Human-readable rule names, reasons, and remediation strings remain in Rust metadata; the kernel receives only fixed-size rule tables and emits integer rule identifiers.

This split keeps the eBPF program verifier-friendly. The kernel program iterates over fixed-size arrays in read-only data using bounded loops, copies each rule entry into a stack-local variable before matching, and uses explicit index bounds checks for argument scanning and pattern comparison.

4.2.2 Kernel State.

The eBPF backend maintains five BPF hash maps:

- `ts_proc`: pid → label bitmask and lineage-gate state.
- `ts_root`, `ts_sess`: root-process and session-level temporal (after) gate state.
- `ts_file`: path hash → label bitmask.
- `ts_endp`: IPv4 address → label bitmask.

BPF map entries are keyed by tid (one entry per Linux thread) for `ts_proc` and by path hash or IPv4 address for object maps. Updates use atomic BPF helper operations; concurrent reads from different CPUs observe a consistent label bitmask per entry.

Tracepoint programs handle process lifecycle events (fork, exec, exit) and post-operation file and connect events. BPF-LSM programs attach to `bprm_check_security`, `file_open`, and `socket_connect` for pre-operation enforcement. When BPF-LSM is not active, the loader uses tracepoint mode and disables block-effect rules. In tracepoint mode, hooks fire after the operation commits; a `connect` tracepoint observes an already-established connection. This is why `effect block`

requires BPF-LSM: only LSM hooks provide a pre-operation verdict. Tracepoint mode supports `audit` (post-operation observation) and `kill` (post-operation task termination).

The current implementation supports up to 32 sources, 32 rules, 16 transforms, and 16 gates per policy. Rule evaluation uses `bpf_loop` for bounded iteration, which the eBPF verifier accepts for policies exceeding 100 rules.

4.2.3 Unified Event Handling. Each hook normalizes kernel arguments into a common event structure: subject pid, object kind, access kind, backend mode, target string, optional argument vector, and optional IPv4 address. A shared handler resolves the subject's label bitmask, applies source labels for read/connect events, evaluates all supported rules, emits at most one `TAINT_VIOLATION` event for the highest-priority matching rule, and then applies propagation updates.

4.3 Corrective Feedback

The kernel violation event contains the rule identifier, effect, target string, pid, and flags indicating whether the operation was blocked or the task was killed. The Rust runner maps the rule identifier to the corresponding rule name, reason, and remediation string, then appends a structured payload to `.actplane/last-violation.txt`.

Agent integration uses a post-tool hook. The `actplane feedback-hook` subcommand reads new bytes from the violation file and returns them as `additionalContext` to compatible agent runtimes [12]. The hook is a read-only relay: it does not re-evaluate policy in userspace. The kernel is the sole authority for rule matches; userspace formats the result so the agent can select an alternative execution path.

5 Implementation

ActPlane is implemented as a Rust driver and an eBPF program pair. The system builds a single `actplane` binary that embeds the compiled eBPF loader, extracts it at runtime, compiles the project's policy DSL, and runs a target command under the enforcement harness.

5.1 Runner and Policy Discovery

The command `actplane run -- <cmd>` discovers `actplane.yaml` by searching upward from the working directory. The policy YAML contains a `policy: |` DSL block. Before the target process begins executing, the loader seeds the stopped target's pid with the `AGENT` label so that all descendants inherit the agent identity. The runner prepares the feedback file (`.actplane/last-violation.txt`) and a hook state file, exports their paths to the child environment, waits for the loader to report readiness, then resumes the target. This sequencing ensures that the first target operation is observed.

5.2 BPF Backends

The eBPF program attaches to process lifecycle tracepoints (fork, exec, exit), syscall tracepoints for file and connect events, and BPF-LSM hooks for pre-operation enforcement. At load time, the loader checks whether the running kernel has the `bpf` LSM active in `/sys/kernel/security/lsm`. If not, it falls back to tracepoint mode, in which the supported effects are `audit` and `kill`; `block` rules are disabled because there is no pre-operation verdict path.

5.3 Limitations

Path identity. File labels are keyed by FNV-1a path hash rather than inode/device pairs. Hard links and mount binds can cause the same file to carry different labels under different paths.

Endpoint identity. Connect matching uses numeric IPv4 addresses. Hostname, DNS, SNI, and HTTP-layer matching require userspace resolution or additional instrumentation.

Argument predicates. The `@arg` predicate inspects arguments after exec via the tracepoint argument buffer. Pre-operation blocking of argument predicates requires an argument cache before `bprm_check_security`, which is not yet implemented.

Label precision. Labels propagate at object granularity. A process that reads one byte from a labeled file acquires the full label set. This can produce over-tainting relative to byte-level systems [13]. Section 6 measures the false-positive rate and validates that declassification and gate paths prevent unnecessary enforcement.

6 Evaluation

The evaluation measures five properties: bypass coverage, semantic-feedback effectiveness, false-positive rate, runtime overhead, and coverage over real-world contracts.

6.1 Experimental Setup

6.2 Bypass Coverage (RQ1)

Does OS-level enforcement detect the same contract violation regardless of the tool path through which the agent triggers it?

We execute the same forbidden operation through five paths: (1) direct tool call, (2) nested shell string (`bash -c`), (3) Python `subprocess`, (4) compiled helper binary, and (5) direct `execve` from generated C code. For each path, we record whether ActPlane detects the violation and whether the semantic feedback payload is delivered to the agent.

6.3 Feedback and Recovery (RQ2)

Does semantic feedback improve agent task completion and reduce repeated violations compared with enforcement without feedback?

We run agent tasks under four conditions: (C1) prompt-only, in which the contract is stated in the prompt but not enforced; (C2) audit, in which ActPlane reports violations but does not fail operations; (C3) enforcement without feedback, in which the operation fails with `-EPERM` or `SIGKILL` but

no structured reason; (C4) enforcement with feedback, in which the operation fails and ActPlane injects the rule name, reason, and remediation into the agent's context.

The primary comparison is C3 vs. C4: the marginal benefit of semantic feedback. The C1-to-C2 comparison measures how often prompt-only contracts are violated. The C2-to-C3 comparison isolates the effect of enforcement from observation.

6.4 False Positives (RQ3)

Do benign tasks and sanctioned contract paths (declassification, gate mediation, temporal gates) proceed without spurious enforcement?

6.5 Overhead (RQ4)

What is the per-event and end-to-end cost of label propagation and rule checking?

We measure per-event latency for fork, exec, open, write, and connect operations with and without ActPlane attached, and report p50 and p99 latencies. We also measure wall-clock time for representative agent tasks and BPF map memory consumption as a function of rule count and active process count.

6.6 Corpus Coverage (RQ5)

How many real-world contracts from the corpus study (§3) are enforceable by ActPlane?

We apply the D1–D7 coding dimensions to each extracted contract and report the fraction that is observable at the syscall boundary (D3), expressible in the ActPlane DSL (D4), enforceable without false positives (D5), and temptable in a realistic agent task (D7). We report inter-rater agreement (Cohen's κ) for each dimension.

7 Related Work

7.1 In-Kernel Provenance and Information Flow

PASS introduced provenance as a storage-layer primitive [16]. Hi-Fi and LPM used LSM hooks to capture trustworthy whole-system provenance [2, 20]. CamFlow extended this to a streaming provenance framework over processes, files, and sockets [19].

CamQuery is the closest prior system [18]. It executes provenance queries inline in the kernel, propagates labels across process, file, and network edges, and can deny operations that violate a policy. ActPlane shares this label-propagation model but differs in three respects: it targets cooperative AI agents rather than adversarial intrusions, compiles an agent-oriented source/sink DSL rather than general provenance queries, and returns semantic feedback to the violating actor. The implementation differs as well: ActPlane uses the eBPF/BPF-LSM substrate rather than a loadable kernel module.

TaintDroid, Dytan, libdft, and Panorama established dynamic taint tracking at runtime, binary-instrumentation, and emulation layers [7, 10, 13, 27]. ActPlane operates at object granularity rather than byte granularity, trading precision for low-overhead whole-system coverage.

7.2 eBPF and LSM Enforcement

BPF-LSM, originally proposed as KRSI, allows eBPF programs to attach to Linux security hooks and return allow/deny verdicts [23, 24]. ActPlane uses BPF-LSM as its pre-operation enforcement backend. Contemporary eBPF enforcement tools such as Tetragon, Tracee, and eBPF-PATROL provide per-event predicate matching and, in Tetragon's case, a boolean lineage flag (`followChildren`) that propagates across fork/exec. These systems evaluate per-event predicates or single-channel lineage flags rather than cross-channel labeled information flow [1, 6, 11].

7.3 Agent Guardrails and Information-Flow Control

AgentSpec and Progent provide deterministic runtime enforcement over tool-call names, arguments, and framework actions [22, 26]. These systems enforce at the tool-call boundary; ActPlane complements them by enforcing below the tool API, where effects that bypass the framework are still visible.

FIDES and CaMeL apply typed information-flow control and capability separation within the agent loop or scaffolding [8, 9]. ActPlane applies the same style of information-flow reasoning at the OS boundary, where enforcement persists across context windows and cannot be circumvented by subprocess indirection.

7.4 Agent Instruction Files

Recent empirical studies characterize `CLAUDE.md`, `AGENTS.md`, and related agent instruction files along dimensions of content taxonomy, maintenance practices, and effect on task efficiency [4, 5, 15, 21]. ActPlane's corpus study (§3) asks a different question: which instructions constitute cross-event information-flow contracts, and which of those can be enforced at the OS level?

8 Conclusion

ActPlane provides a programmable OS-level control plane for agent harnesses. It compiles agent-declared behavioral contracts into eBPF/BPF-LSM labeled information-flow rules, propagates labels across process, file, and endpoint objects at kernel hooks, and returns kernel-detected violations as semantic feedback to the agent. By supporting audit, pre-operation denial, and harness-level termination with structured remediation, ActPlane enables agents to recover from contract violations rather than fail opaquely. ActPlane unifies labeled information-flow observation and enforcement with an agent-oriented contract language and a feedback loop on

the eBPF/BPF-LSM substrate, bridging the intent–behavior gap that single-level harnesses leave open.

References

[1] Aqua Security. 2026. Tracee: Linux Runtime Security and Forensics using eBPF. https://github.com/aquasecurity/tracee.

[2] Adam Bates, Dave Tian, Kevin R. B. Butler, and Thomas Moyer. 2015. Trustworthy Whole-System Provenance for the Linux Kernel. In *24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, Washington, DC, 319–334. https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/bates

[3] Birgitta Boeckeler. 2026. Harness Engineering for Coding Agent Users. https://martinfowler.com/articles/harness-engineering.html.

[4] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. 2025. Agent READMEs: An Empirical Study of Context Files for Agentic Coding. arXiv:2511.12884. https://arxiv.org/abs/2511.12884

[5] Worawalan Chatlatanagulchai, Kundjanasith Thonglek, Brittany Reid, Yutaro Kashiwa, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, and Hajimu Iida. 2025. On the Use of Agentic Coding Manifests: An Empirical Study of Claude Code. arXiv:2509.14744. https://arxiv.org/abs/2509.14744

[6] Cilium Project. 2026. Tetragon: eBPF-based Security Observability and Runtime Enforcement. https://tetragon.io/.

[7] James Clause, Wanchun Li, and Alessandro Orso. 2007. Dytan: A Generic Dynamic Taint Analysis Framework. In *Proceedings of the 2007 International Symposium on Software Testing and Analysis*. Association for Computing Machinery, London, United Kingdom, 196–206. doi:10.1145/1273463.1273490

[8] Manuel Costa, Boris Koepf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Beguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643. https://arxiv.org/abs/2505.23643

[9] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramer. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813. https://arxiv.org/abs/2503.18813

[10] William Enck, Peter Gilbert, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. 2010. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*. USENIX Association, Vancouver, Canada, 393–407. https://www.usenix.org/conference/osdi10/taintdroid-information-flow-tracking-system-realtime-privacy-monitoring

[11] Sangam Ghimire, Nirjal Bhurtel, Roshan Sahani, and Sudan Jha. 2025. eBPF-PATROL: Protective Agent for Threat Recognition and Overreach Limitation using eBPF in Containerized and Virtualized Environments. arXiv:2511.18155. https://arxiv.org/abs/2511.18155

[12] Yuxuan Jiang, Meng Xu, Yiwen Song, and Xinwen Fu. 2025. AgentSight: Bridging the Semantic Gap Between Agent Intent and Low-Level Actions. arXiv:2508.02736. https://arxiv.org/abs/2508.02736

[13] Vasileios P. Kemerlis, Georgios Portokalidis, Kangkook Jee, and Angelos D. Keromytis. 2012. libdft: Practical Dynamic Data Flow Tracking for Commodity Systems. In *Proceedings of the 8th ACM SIGPLAN/SIGOPS Conference on Virtual Execution Environments*. Association for Computing Machinery, London, United Kingdom, 121–132. doi:10.1145/2151024.2151042

[14] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. In *Transactions of the Association for Computational Linguistics*, Vol. 12. 157–173.

[15] Jai Lal Lulla, Seyedmoein Mohsenimofidi, Matthias Galster, Jie M. Zhang, Sebastian Baltes, and Christoph Treude. 2026. On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents. arXiv:2601.20404. https://arxiv.org/abs/2601.20404

[16] Kiran-Kumar Muniswamy-Reddy, David A. Holland, Uri Braun, and Margo Seltzer. 2006. Provenance-Aware Storage Systems. In *2006 USENIX Annual Technical Conference (USENIX ATC 06)*. USENIX Association, Boston, MA, 43–56. https://www.usenix.org/conference/2006-usenix-annual-technical-conference/provenance-aware-storage-systems

[17] OpenBSD Project. 2015. pledge(2) – restrict system operations. https://man.openbsd.org/pledge.2.

[18] Thomas F. J.-M. Pasquier, Xueyuan Han, Thomas Moyer, Adam Bates, Olivier Hermant, David Eyers, Jean Bacon, and Margo Seltzer. 2018. Runtime Analysis of Whole-System Provenance. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, Toronto, Canada, 1601–1616. doi:10.1145/3243734.3243776

[19] Thomas F. J.-M. Pasquier, Jatinder Singh, David Eyers, and Jean Bacon. 2017. CamFlow: Managed Data-Sharing for Cloud Services. *IEEE Transactions on Cloud Computing* 5, 3 (2017), 472–484. doi:10.1109/TCC.2015.2489211

[20] Devin J. Pohly, Stephen McLaughlin, Patrick McDaniel, and Kevin Butler. 2012. Hi-Fi: Collecting High-Fidelity Whole-System Provenance. In *Proceedings of the 28th Annual Computer Security Applications Conference*. Association for Computing Machinery, Orlando, FL, 259–268. doi:10.1145/2420950.2420989

[21] Helio Victor F. Santos, Vitor Costa, Joao Eduardo Montandon, and Marco Tulio Valente. 2025. Decoding the Configuration of AI Coding Agents: Insights from Claude Code Projects. arXiv:2511.09268. https://arxiv.org/abs/2511.09268

[22] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703. https://arxiv.org/abs/2504.11703

[23] KP Singh. 2019. Kernel Runtime Security Instrumentation. Linux Plumbers Conference. https://lpc.events/event/4/contributions/454/

[24] The Linux Kernel Documentation. 2021. LSM BPF Programs. https://www.kernel.org/doc/html/v5.15/bpf/bpf_lsm.html.

[25] Vivek Trivedy. 2026. The Anatomy of an Agent Harness. https://www.langchain.com/blog/the-anatomy-of-an-agent-harness.

[26] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv:2503.18666. https://arxiv.org/abs/2503.18666

[27] Heng Yin, Dawn Song, Manuel Egele, Christopher Kruegel, and Engin Kirda. 2007. Panorama: Capturing System-wide Information Flow for Malware Detection and Analysis. In *Proceedings of the 14th ACM Conference on Computer and Communications Security*. Association for Computing Machinery, Alexandria, VA, 116–127. doi:10.1145/1315245.1315261